



南開大學  
Nankai University

计算机学院  
人工智能导论实验报告

实验二：黑白棋

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2025 年 4 月 12 日

# 目录

|                                |           |
|--------------------------------|-----------|
| <b>1 问题重述</b>                  | <b>2</b>  |
| <b>2 设计思想</b>                  | <b>2</b>  |
| 2.1 蒙特卡洛树搜索算法的原理 . . . . .     | 2         |
| 2.2 算法设计 . . . . .             | 3         |
| 2.2.1 选择 . . . . .             | 3         |
| 2.2.2 扩展 . . . . .             | 3         |
| 2.2.3 模拟 . . . . .             | 4         |
| 2.2.4 反向传播 . . . . .           | 4         |
| 2.2.5 蒙特卡洛树构建及返回最优棋步 . . . . . | 5         |
| <b>3 代码内容</b>                  | <b>6</b>  |
| <b>4 实验结果</b>                  | <b>9</b>  |
| <b>5 总结</b>                    | <b>11</b> |

## 1 问题重述

1. 黑方先行，双方交替下棋。
  2. 一步合法的棋步包括：  
在一个空格新落下一个棋子，并且翻转对手一个或多个棋子；  
新落下的棋子必须落在可夹住对方棋子的位置上，对方被夹住的所有棋子都要翻转过来，可以是横着夹，竖着夹，或是斜着夹。  
夹住的位置上必须全部是对手的棋子，不能有空格；  
一步棋可以在数个（横向，纵向，对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不翻某个棋子。
  3. 如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。
  4. 如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。
  5. 棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。
  6. 如果某一方落子时间超过 1 分钟 或者 连续落子 3 次不合法，则判该方失败。
- 使用『蒙特卡洛树搜索算法』实现 miniAlphaGo for Reversi

对于上面的问题，我们需要抓住几个关键点，首先第一个是要获得合法的棋步，可以通过 board 类中的 get\_legal\_actions 方法获得，另外要抓住的一个关键信息是对于棋局结束的判断：直到棋盘填满或者双方都无合法棋步可下，另外还有一些信息，比如说每棋步思考不能超过 1 分钟，不能连续 3 次落子不合法，否则就会判定失败。

## 2 设计思想

基于问题，我们需要使用蒙特卡洛树搜索算法去设计一个类似于 miniAlphaGo for Reversi 的 AI 玩家。首先需要了解一下蒙特卡洛树搜索算法的原理：

### 2.1 蒙特卡洛树搜索算法的原理

蒙特卡洛树搜索通过迭代执行选择、扩展、模拟和反向传播四个步骤，逐步优化决策路径，适用于复杂博弈和决策问题。其核心优势在于能够通过大量随机模拟探索可能的决策路径，帮助 AI 做出更明智的选择。

**1. 选择 (Selection)** 从搜索树的根节点（当前状态）开始，根据某种策略（如 UCB1 公式）递归选择最优的子节点，直到到达一个未被完全扩展的节点。

**2. 扩展 (Expansion)** 当选择的节点未被完全扩展时，随机选择一个未被访问过的子节点并将其添加到搜索树中。这一步骤增加了搜索树的规模，使得算法能够探索更多的可能行动。

**3. 模拟 (Simulation)** 从新扩展的节点开始，进行一次随机模拟游戏，直到游戏结束。在模拟过程中，每个行动都是随机选择的。通过模拟游戏，得到一个最终的奖励结果（如胜利、失败或平局对应的奖励值）。

**4. 反向传播 (Backpropagation)** 将模拟得到的奖励结果沿着从新扩展的节点到根节点的路径进行反向传播，更新路径上每个节点的访问次数和平均奖励。这一步骤使得搜索树中的节点信息能够反映出该节点的潜在价值。

## 2.2 算法设计

我们基于蒙特卡洛树搜索算法的四个步骤给出部分代码。

### 2.2.1 选择

```

1  def selection(self, weight):
2      cur_Node=self
3      while cur_Node.children and not cur_Node.untried_actions:
4          if random.random() > self.epsilon:
5              cur_Node = cur_Node.best_child()
6          else:
7              cur_Node = random.choice(cur_Node.children)
8              self.epsilon *= self.gamma
9      return cur_Node

```

这是选择过程的逻辑，如果没有合法的 action，就说明没有未扩展的子节点，那我们就一直往下选择。我们设定了一个概率值 epsilon，还有一个用于减小 epsilon 的因子 gamma，蒙特卡洛树开始构建的时候，epsilon 比较大，所以在选择时会偏向使用 random.choice，偏向探索，随着算法一次次执行，偏向使用 best\_child，偏向利用。

```

1  def best_child(self, weight_c=1.414):
2      # 防止除零错误
3      ucb1=[]
4      for child in self.children:
5          if child.visits == 0:
6              ucb1.append(float('inf')) # 未访问的节点给予最高优先级
7          else:
8              # 使用更优的UCB参数
9              exploitation = child.wins / child.visits
10             exploration = weight_c * math.sqrt(2*math.log(self.visits) /
11                 child.visits)
12             ucb1.append(exploitation + exploration)
13     return self.children[np.argmax(ucb1)]

```

这是我们在选择过程中调用的函数，其实就是计算 ucb 值，返回最大 ucb 值的孩子节点。需要注意的是，在最开始的时候，由于 visits=0，我们要优先选择未探索的节点，给予最高优先级。

### 2.2.2 扩展

```

1  def expansion(self):
2      if self.untried_actions:
3          action=random.choice(self.untried_actions)
4          return self.add_child(action)

```

```
5         return self
```

在扩展阶段，我们判断，如果这个节点有合法的行动，即其具有未扩展的节点，我们就进行一次扩展，创建一个子节点，为其随机选择一个行动。

```
1     def add_child(self, action):
2         temp_board=copy.deepcopy(self.board)
3         temp_board._move(action, self.color)
4         child_color='O' if self.color=='X' else 'X'
5         child=MCTSNode(temp_board, child_color, self, action)
6         self.children.append(child)
7         self.untried_actions.remove(action)
8     return child
```

扩展阶段需要调用 add\_child 函数，来创建子节点。注意我们构建蒙特卡洛树时，不能对原来的棋盘进行操作，所以我们需要复制新的棋盘。另外注意，因为这是一个黑白棋对弈游戏，所以父节点和子节点的角色不同。注意在扩展了子节点之后，在父节点未扩展的子节点中把此步操作删除掉。

### 2.2.3 模拟

```
1     def simulation(self):
2         flag=0
3         temp_board=copy.deepcopy(self.board)#在复制的棋盘上进行模拟
4         cur_color=self.color
5         while True:
6             if temp_board.count('X')+temp_board.count('O')== 64 or flag==2:
7                 winner, diff=temp_board.get_winner()
8                 return winner, diff #返回模拟胜者
9             legal_actions=list(temp_board.get_legal_actions(cur_color))
10            if not legal_actions:
11                flag+=1
12                cur_color='O' if cur_color=='X' else 'X'
13            else:
14                action=random.choice(legal_actions)
15                temp_board._move(action, cur_color)
16                cur_color='O' if cur_color=='X' else 'X'
```

在模拟阶段，我们针对扩展出来的那个节点，继续游戏直至游戏结束。需要注意终局的游戏判断是，棋盘满了或者双方都无棋步可走，为此我定义了一个 flag 标志，用于判断双方都无棋步可走，并调用 board 类中的 get\_winner，返回模拟的胜者及赢了多少颗棋子。

### 2.2.4 反向传播

```
1     def backpropagation(self, result, diff):
2         cur=self
3         while cur:
4             cur.update(result, diff)
5             cur=cur.parent
```

反向传播阶段需要从结果反向传播到根节点，调用 update 函数对路径上每个节点的 wins 和 visits 进行更新。

```
1 def update(self, result, diff):
2     self.visits+=1
3     new_value=10+diff
4     if result==1:
5         self.wins+=new_value if self.color=='X' else -(new_value)
6     elif result==0:
7         self.wins+=new_value if self.color=='O' else -(new_value)
8     else:
9         pass #平局
```

选择收益为胜利得 10 分，每多赢一个棋子加 1 分，对于胜者加分，对于败者减分。

### 2.2.5 蒙特卡洛树构建及返回最优棋步

```
1 def MCTS_search(self, iterations=1000000, time_limit=60):
2     start=time.time()
3     cnt=0
4     empty_count = sum(1 for i in range(8) for j in range(8) if
5         self.board._board[i][j] == '.')
6
7     # 根据游戏阶段调整探索/利用平衡
8     if empty_count > 40: # 游戏早期，多探索
9         exploration_weight = 2.5 # 大幅增加探索权重
10    elif empty_count > 20: # 游戏中期，平衡探索和利用
11        exploration_weight = 1.5 # 增加探索
12    else: # 游戏后期，更注重利用
13        exploration_weight = 0.3 # 更注重利用
14
15    while cnt<iterations:
16        if time.time()-start>time_limit:
17            break #超时
18
19        node=self.selection(exploration_weight)
20
21        if node.untried_actions:
22            node=node.expansion()
23
24        result, diff=node.simulation()
25
26        node.backpropagation(result, diff)
27        cnt+=1
28
29    return self.get_best_child(weight_c=exploration_weight).action
```

此阶段进行蒙特卡洛树构建的 4 个步骤，并限制了其思考时间。此外，还根据棋局状态对探索因子进行修改，前期偏向探索，所以设置大一点，后期偏向利用，所以设置小一点。然后注意的是，在返回最优棋步是，调用的是 `get_best_child` 函数，这个函数不同于 `best_child` 就是返回的是 `wins/visits` 值最大的 `action`，因为最后选择最优棋步是只注重利用。

```

1  def get_best_child(self, weight_c=1.414):
2      # 防止除零错误
3      ucb1=[]
4      for child in self.children:
5          if child.visits == 0:
6              ucb1.append(float('inf')) # 未访问的节点给予最高优先级
7          else:
8              # 使用更优的UCB参数
9              ucb1.append(child.wins/child.visits)
10     return self.children[np.argmax(ucb1)]

```

### 3 代码内容

```

1  import math
2  import numpy as np
3  import random
4  import copy
5  import time
6
7  class MCTSNode:
8      def __init__(self, board, color, parent=None, action=None):
9          self.board=board
10         self.color=color
11         self.parent=parent
12         self.children=[]
13         self.visits=0
14         self.wins=0
15         self.action=action
16         self.epsilon=0.3
17         self.gamma=0.999
18         self.untried_actions=list(board.get_legal_actions(color))
19
20     def add_child(self, action):
21         temp_board=copy.deepcopy(self.board)
22         temp_board._move(action, self.color)
23         child_color='O' if self.color=='X' else 'X'
24         child=MCTSNode(temp_board, child_color, self, action)
25         self.children.append(child)
26         self.untried_actions.remove(action)
27         return child
28
29

```

```

30 def best_child(self, weight_c=1.414):
31     # 防止除零错误
32     ucb1=[]
33     for child in self.children:
34         if child.visits == 0:
35             ucb1.append(float('inf')) # 未访问的节点给予最高优先级
36         else:
37             # 使用更优的UCB参数
38             exploitation = child.wins / child.visits
39             exploration = weight_c * math.sqrt(2*math.log(self.visits) /
40                 child.visits)
41             ucb1.append(exploitation + exploration)
42     return self.children[np.argmax(ucb1)]
43
44 def get_best_child(self, weight_c=1.414):
45     # 防止除零错误
46     ucb1=[]
47     for child in self.children:
48         if child.visits == 0:
49             ucb1.append(float('inf')) # 未访问的节点给予最高优先级
50         else:
51             # 使用更优的UCB参数
52             ucb1.append(child.wins/child.visits)
53     return self.children[np.argmax(ucb1)]
54
55 def update(self, result, diff):
56     self.visits+=1
57     new_value=10+diff
58     if result==1:
59         self.wins+=new_value if self.color=='X' else -(new_value)
60     elif result==0:
61         self.wins+=new_value if self.color=='O' else -(new_value)
62     else:
63         pass #平局
64
65 def selection(self, weight):
66     cur_Node=self
67     while cur_Node.children and not cur_Node.untried_actions:
68         if random.random() > self.epsilon:
69             cur_Node = cur_Node.best_child()
70         else:
71             cur_Node = random.choice(cur_Node.children)
72     self.epsilon *= self.gamma
73     return cur_Node
74
75 def expansion(self):
76     if self.untried_actions:
77         action=random.choice(self.untried_actions)

```



```

78         return self.add_child(action)
79     return self
80
81
82     def simulation(self):
83         flag=0
84         temp_board=copy.deepcopy(self.board)#在复制的棋盘上进行模拟
85         cur_color=self.color
86         while True:
87             if temp_board.count('X')+temp_board.count('O')== 64 or flag==2:
88                 winner,diff=temp_board.get_winner()
89                 return winner,diff #返回模拟胜者
90             legal_actions=list(temp_board.get_legal_actions(cur_color))
91             if not legal_actions:
92                 flag+=1
93                 cur_color='O' if cur_color=='X' else 'X'
94             else:
95                 action=random.choice(legal_actions)
96                 temp_board._move(action,cur_color)
97                 cur_color='O' if cur_color=='X' else 'X'
98
99     def backpropagation(self,result,diff):
100         cur=self
101         while cur:
102             cur.update(result,diff)
103             cur=cur.parent
104
105     def MCTS_search(self,iterations=1000000,time_limit=60):
106         start=time.time()
107         cnt=0
108         empty_count = sum(1 for i in range(8) for j in range(8) if
109                             self.board._board[i][j] == '.')
110
111         # 根据游戏阶段调整探索/利用平衡
112         if empty_count > 40: # 游戏早期, 多探索
113             exploration_weight = 2.5 # 大幅增加探索权重
114         elif empty_count > 20: # 游戏中期, 平衡探索和利用
115             exploration_weight = 1.5 # 增加探索
116         else: # 游戏后期, 更注重利用
117             exploration_weight = 0.3 # 更注重利用
118
119         while cnt<iterations:
120             if time.time()-start>time_limit:
121                 break #超时
122
123             node=self.selection(exploration_weight)
124
125             if node.untried_actions:
126                 node=node.expansion()

```

```

126         result , diff=node.simulation()
127
128
129         node.backpropagation(result , diff)
130         cnt+=1
131
132     return self.get_best_child(weight_c=exploration_weight).action
133
134
135 class AIPlayer:
136     """
137     AI 玩家
138     """
139
140     def __init__(self, color):
141         """
142         玩家初始化
143         :param color: 下棋方, 'X' - 黑棋, 'O' - 白棋
144         """
145
146         self.color = color
147
148     def get_move(self, board):
149         """
150         根据当前棋盘状态获取最佳落子位置
151         :param board: 棋盘
152         :return: action 最佳落子位置, e.g. 'A1'
153         """
154         if self.color == 'X':
155             player_name = '黑棋'
156         else:
157             player_name = '白棋'
158         print("请等一会, 对方 {}-{} 正在思考中...".format(player_name, self.color))
159
160         # -----请实现你的算法代码-----
161         action = MCTSNode(board, self.color).MCTS_search(time_limit=30)
162         # -----
163
164     return action

```

## 4 实验结果

1. 黑棋先手: 领先 46 个棋子。

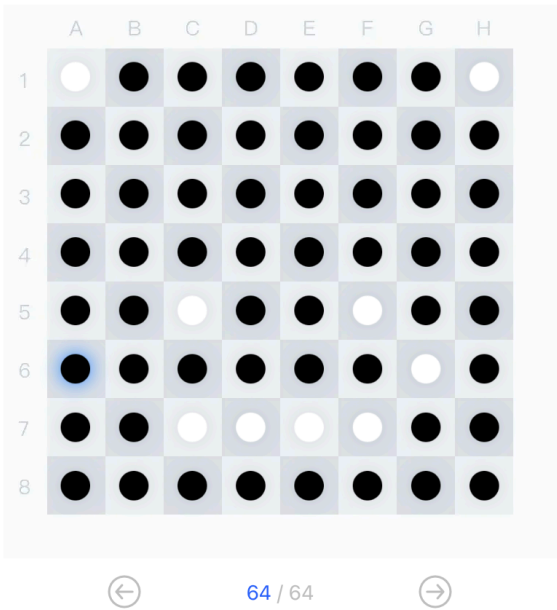
测试详情 展示棋盘

| 测试点  | 状态 | 时长   | 结果              |
|------|----|------|-----------------|
| 对手对弈 | ✓  | 959s | 黑棋获胜, 领先棋子数: 46 |

确定

图 4.1: 黑棋先手

测试详情 隐藏棋盘



棋局胜负: 黑棋赢

先后手: 黑棋先手

棋局难度: 高级

当前棋子: 黑棋

当前坐标: A6

图 4.2: 黑棋先手

2. 白棋后手: 领先 28 个棋子。

测试详情 展示棋盘

| 测试点  | 状态 | 时长   | 结果              |
|------|----|------|-----------------|
| 对手对弈 | ✓  | 169s | 白棋获胜, 领先棋子数: 28 |

确定

图 4.3: 白棋后手

## 测试详情 隐藏棋盘 ^



图 4.4: 白棋后手

## 5 总结

1. 加强了对 python 中类、函数、参数的使用与代码书写。
2. 更好地理解了蒙特卡洛树的四个步骤并一一实现出来。
3. 加强了一点点探索问题的能力，比如最开始的时候，在选择和返回最好的 action 的时候，我利用的都是 ucb 值，但之后发现在最后抉择的时候，只注重利用，所以探索因子应为 0。